

****Gouvernance des Projets Open Source****

:::

Session 8 – Cours M1 Introduction aux Logiciels Libres

****Objectifs de la séance****

****Ce que vous saurez faire****

À la fin de cette séance, vous serez capables de :

- **Identifier** les modèles de gouvernance des projets majeurs
- **Expliquer** le droit au fork et ses implications
- **Analyser** les mécanismes de prise de décision
- **Comprendre** le parcours pour devenir mainteneur
- **Connaître** le rôle d'un OSP0 en entreprise

****Actualité : Euro Office vs OnlyOffice (mars 2026)****

Les faits : Une coalition de 8 éditeurs européens (Ionos, Nextcloud, XWiki, OpenProject, Abilian...) lance **Euro Office**, fork de la suite OnlyOffice sous AGPL v3. OnlyOffice crie à la violation de licence.

L'argument d'OnlyOffice : Leurs conditions supplémentaires (Section 7 AGPL) exigent de conserver le logo et la marque OnlyOffice. Le fork les a retirés → violation.

Sources :

<https://itsocial.fr/logiciel-agilite/logiciel-agilite-actualites/une-coalition-de-huit-editeurs-europeens-lance-euro-office-alternative-bureautique-souveraine-et-open-source/> –

<https://www.onlyoffice.com/blog/2026/03/onlyoffice-flags-license-violations-in-euro-office-project-by-nextcloud-and-ionos>

****Euro Office vs OnlyOffice : pourquoi l'argument ne tient pas****

Ce que la Section 7 AGPL autorise réellement :

- **7(b)** : exiger l'**attribution** (créditer l'auteur) – pas forcer l'affichage d'une marque dans le produit
- **7(e)** : **refuser** les droits sur les marques → Euro Office ne peut PAS utiliser la marque OnlyOffice (ce qu'ils font)

Et pour les conditions qui vont au-delà ?

"All other non-permissive additional terms are considered 'further restrictions' within the meaning of section 10. If the Program as you received it [...] contains a notice stating that it is governed by this License along with a term that is a further restriction, **you may remove that term.**" – AGPL v3, Section 7

Conclusion : 1) droit d'auteur ≠ droit des marques. 2) On ne peut pas prétendre faire du logiciel libre et refuser un fork. ⇒ Le fork est légitime.

****Actualité : TDF expulse les développeurs de Collabora (avril 2026)****

Les faits : The Document Foundation (TDF, fondation derrière LibreOffice) expulse **30+ membres** affiliés à Collabora, dont **7 des 10 principaux committers** et plusieurs fondateurs du projet.

Les griefs de Collabora (Michael Meeks) :

- Le board a été rempli de **non-techniciens**
- Les développeurs qui écrivent le code sont exclus de la **gouvernance**
- Application sélective des **marques** (contre Collabora, pas contre d'autres)
- Manipulations d'**élections** et de statuts

La réponse de Collabora :

- Infrastructure de code review **indépendante** (gerrit)
- Divergence accrue entre **Collabora Office** et LibreOffice upstream
- Signal clair vers un **fork de facto**

Source : <https://www.collaboraonline.com/blog/tdf-ejects-its-core-developers/>

****Partie 1****

Le droit au fork

****Le fork : garantie fondamentale****

"The indispensable ingredient that binds developers together on a free software project is the code's forkability." – Karl Fogel, *Producing Open Source Software*

Définition : Droit de copier un projet et de le développer indépendamment.

Ce qui le rend possible :

- Licence libre
- Code source disponible
- Aucune autorisation nécessaire

****Pourquoi le fork est fondamental****

Pour la liberté

- Exit option
- Indépendance vis-à-vis du mainteneur
- Protection contre les dérives
- Pérennité du projet

Pour la gouvernance

- "Épée de Damoclès"
- Force à écouter la communauté
- Limite le pouvoir des "dictateurs"
- Mécanisme de dernier recours

****Le paradoxe du fork****

■ "The possibility of forks is usually a much greater force than actual forks."

Les vrais forks sont rares. Pourquoi ?

Coûts d'un fork

- Division de la communauté
- Duplication des efforts
- Confusion pour les utilisateurs
- Perte de momentum

Effet dissuasif

- La menace suffit souvent
- Incite au compromis
- Protège la communauté
- Responsabilise les mainteneurs

****Forks célèbres****

Fork	Original	Raison	Succès ?
LibreOffice	OpenOffice	Rachat par Oracle	✓ Oui
MariaDB	MySQL	Rachat par Oracle	✓ Oui
io.js	Node.js	Gouvernance	✓ (réunification)
Jenkins	Hudson	Conflit Oracle	✓ Oui
OpenTofu	Terraform	Changement licence	En cours
Valkey	Redis	Changement licence	En cours

****Partie 2****

Modèles de gouvernance

****Le modèle BDFL****

BDFL = Benevolent Dictator For Life

Caractéristiques

- Une personne a le dernier mot
- Autorité basée sur le mérite
- Délégation possible
- "Dictature" tempérée par le fork

Exemples

- Linux : Linus Torvalds
- Python : Guido van Rossum (ex-)
- Ubuntu : Mark Shuttleworth

****Le BDFL en pratique****

"Only when it is clear that no consensus can be reached, and that most of the group wants someone to make a decision so that development can move on, does she put her foot down."

Le bon BDFL :

- Décide rarement par autorité
- Écoute la communauté
- Utilise son pouvoir en dernier recours
- Peut se retirer (comme Guido)

****Le modèle Apache (méritocratie)****

Structure en cercles :



****Méritocratie Apache : Détails****

Rôle	Droits	Comment l'obtenir
Utilisateur	Utiliser, reporter bugs	Automatique
Contributeur	Proposer des patches	Automatique
Committer	Commit direct	Invitation par le PMC
PMC Member	Vote, décisions	Invitation par le PMC
PMC Chair	Représentation officielle	Élection

Principe : Les droits s'obtiennent par le mérite (contributions).

****Le modèle comité élu****

Exemples modernes : Rust, Python (post-Guido), Node.js

Caractéristiques

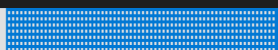
- Steering Committee élu
- Mandats limités
- Processus formalisé (RFC)
- Transparence

Avantages

- Légitimité démocratique
- Renouvellement
- Évite la concentration
- Processus clairs

****Comparatif des modèles****

Aspect	BDFL	Méritocratie	Comité élu
Décisions	Rapides	Consensus	Formalisées
Succession	Difficile	Naturelle	Organisée
Scalabilité	Moyenne	Bonne	Bonne
Transparence	Variable	Moyenne	Haute
Exemples	Linux	Apache	Rust

****Partie 3******Prise de décision**

****Le consensus paresseux (Lazy Consensus)****

"As long as nobody explicitly opposes a proposal, it is recognized as having the support of the community."

Fonctionnement :

1. Une proposition est faite
2. Période d'attente
3. Si personne ne s'oppose → accepté
4. Si objection → discussion

Avantage : Efficace, évite la bureaucratie.

****Quand le consensus échoue******Mécanismes de résolution :**

1. **Discussion prolongée** – Chercher un compromis
2. **Vote** – Majorité qualifiée (souvent 2/3 ou 3/4)
3. **Décision BDFL/PMC** – Autorité tranche
4. **Report** – Attendre plus d'information
5. **Fork** – Dernier recours

****Les RFC / PEP / CEP****

Processus formalisé pour les changements majeurs.

Projet	Nom	Description
Python	PEP	Python Enhancement Proposal
Rust	RFC	Request for Comments
Kubernetes	KEP	Kubernetes Enhancement Proposal
IETF	RFC	Internet standards

Contenu type : Motivation, design, alternatives, impact

Comparable aux ADR (Architecture Decision Records) utilisés en entreprise – même logique de documenter les décisions et leurs justifications, mais les PEP/RFC sont publics et soumis à la communauté.

****Exemple : Processus PEP****

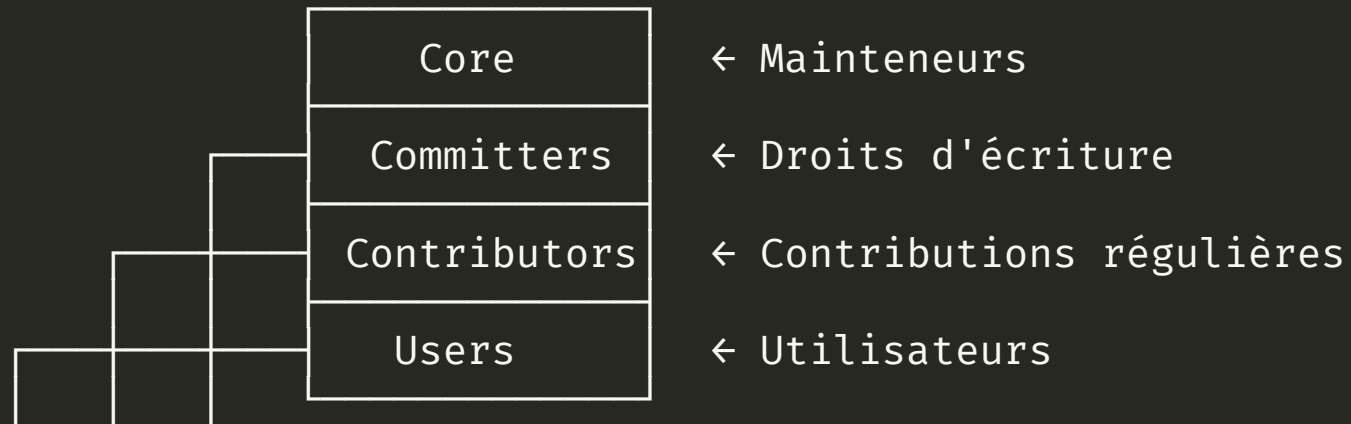
1. Idée → Discussion informelle
2. Draft PEP → Rédaction formelle
3. Review → Commentaires de la communauté
4. Décision → Accepté / Rejeté / Différé
5. Implémentation → Si accepté
6. Final → Documentation officielle

****Types de PEP :****

- **Standards Track** – Changements techniques (nouveau feature, API) – ex: PEP 484 (type hints)
- **Informational** – Recommandations, bonnes pratiques – ex: PEP 8 (style guide)
- **Process** – Changements de processus – ex: PEP 13 (gouvernance post-Guido)

****Partie 4****

Devenir mainteneur

****Le modèle "Onion"****

Progression : De l'extérieur vers l'intérieur, par le mérite.

****Comment devenir committer/mainteneur******Ce qui compte**

- Contributions régulières et qualité
- Revue de code des autres
- Aide aux nouveaux
- Participation aux discussions
- Fiabilité et constance

Le processus

- Généralement par cooptation
- Proposition par un mainteneur existant
- Vote ou consensus du groupe
- Période d'essai parfois

****Responsabilités d'un mainteneur****

Techniques

- Revoir les PRs
- Merger le code
- Maintenir la qualité
- Gérer les releases
- Corriger les bugs critiques

Communautaires

- Accueillir les nouveaux
- Modérer les discussions
- Communiquer les décisions
- Représenter le projet

****Le burnout des mainteneurs****

Problème croissant dans l'open source.

Causes

- Charge de travail souvent / parfois bénévole
- Pression des utilisateurs
- Critiques non constructives
- Solitude

Solutions

- Délégation
- Dire non
- Prendre des pauses
- Chercher des co-mainteneurs

****Partie 5******Les fondations**

****Pourquoi des fondations ?****

Besoins des projets

- Entité légale
- Détention de marques/domaines
- Réception de dons (defiscalisation?)
- Protection juridique

Ce qu'elles apportent

- Neutralité (pas une entreprise)
- Pérennité
- Infrastructure
- Gouvernance formelle (parfois lourde)

****Fondations majeures****

Fondation	Style	Projets notables
Apache SF	Communautaire	Kafka, Spark, Hadoop
Linux Foundation	Corporate	Linux, Kubernetes, Node
Mozilla	Mission-driven	Firefox
Eclipse	Corporate	Eclipse IDE, Jakarta
CNCF	Corporate	Kubernetes, Prometheus
Python SF	Communautaire	Python, PyPI
FSF	Idéologique	GNU, GCC

****Apache vs Linux Foundation****

Apache Software Foundation

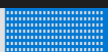
- "Community over code"
- Projets doivent suivre "Apache Way"
- Gouvernance uniforme
- 99% bénévole (staff réduit)

Linux Foundation

- Modèle "umbrella"
- Projets gardent leur gouvernance
- Financé par les entreprises, budget énorme (500 M\$)
- Staff important

****Partie 6****

L'OSPO (Open Source Program Office)



****Qu'est-ce qu'un OSP0 ?****

Open Source Program Office – Structure dédiée à la gestion de l'open source en entreprise.

Missions principales

- Politique d'utilisation
- Conformité licences
- Contribution upstream
- Relations communautaires

Rattachement

- CTO / Direction technique
- Direction juridique
- Parfois indépendant
- Transverse aux équipes

****Pourquoi créer un OSP0 ?****

Enjeux pour l'entreprise :

1. **Juridiques** – Conformité aux licences, gestion des risques
2. **Sécurité** – Supply chain, vulnérabilités
3. **Stratégiques** – Influence, recrutement, innovation
4. **Économiques** – Optimisation des coûts, éviter le vendor lock-in

■ Les entreprises utilisent en moyenne 500+ composants open source.

****Les responsabilités d'un OSP0******Consommation**

- Inventaire des dépendances
- Validation des licences
- Gestion des vulnérabilités
- Formation des développeurs

Contribution

- Politique de contribution
- Process de validation
- Relations avec les projets
- Sponsoring / donations

****Entreprises avec un OSP0****

Entreprise	OSP0 depuis	Focus
Google	2004	Innovation, Android, Kubernetes
Microsoft	2014	Transformation, GitHub
Red Hat	Origine	Core business
SAP	2017	Conformité, contribution
Spotify	2018	InnerSource, outils
SNCF	2020	Souveraineté

TODO Group (Linux Foundation) : communauté de 80+ OSP0 – <https://todogroup.org/>

****Résumé****

****Ce qu'il faut retenir****

1. **Fork** : Garantie fondamentale, rarement exercée mais toujours présente
2. **Modèles** : BDFL, méritocratie Apache, comités élus
3. **Décisions** : Consensus paresseux, votes, RFC/PEP
4. **Progression** : Modèle onion, cooptation par le mérite
5. **Fondations** : Structures légales neutres pour les projets
6. **OSPO** : Structure clé pour gouverner l'open source en entreprise

****Pour la prochaine séance****

Séance 9 : Modèles économiques et open source en entreprise

Préparation suggérée :

- Comment Red Hat gagne-t-il de l'argent avec des logiciels "gratuits" ?
- Qu'est-ce que l'InnerSource ?

****Questions ?****

⋮

Passons au TD